FETCH API – Daily Assignment

1. Problem Statement

The objective of this assignment is to develop the frontend functionality of a blogging platform where users can view blog posts and manage their tasks using a todo list feature. The application must retrieve blog posts and todo items from remote servers using the Fetch API.

The application should:

- Asynchronously fetch blog post data from the `/posts` endpoint.
- Asynchronously fetch todo list data from the `/todos` endpoint.
- Dynamically display the fetched data on the web page using DOM manipulation.
- Implement proper error handling for network failures or unexpected server responses.
- Refactor the code using a suitable JavaScript design pattern to improve maintainability and scalability.
- Integrate all features into a cohesive and well-structured application.

---

2. Objective

The primary objective of this assignment is to gain practical experience in:

- Working with the Fetch API
- Handling JSON data
- Manipulating the DOM dynamically
- Implementing error handling mechanisms
- Applying JavaScript design patterns
- Structuring scalable frontend applications

---

3. Approach

The project was implemented in the following steps:

1. Designed a structured HTML layout containing:
    - Blog Posts section
    - Todo List section

2. Applied CSS styling to ensure clean layout and readability.
3. Used the Fetch API with async/await to retrieve data from remote servers.
4. Converted the server response to JSON using:
   response.json()
5. Dynamically rendered posts and todos using DOM manipulation methods:
   - createElement()
   - appendChild()
   - innerHTML
6. Implemented proper error handling using:
   - try/catch blocks
   - response.ok validation
   - Displaying user-friendly error messages in the UI
7. Refactored the code using the Module Pattern to:
   - Avoid global variables
   - Improve code organization
   - Enhance scalability and maintainability

---

4. Technologies Used

- HTML5
- CSS3
- JavaScript (ES6)
- Fetch API
- JSON
- Module Pattern (JavaScript Design Pattern)
- JSONPlaceholder Public API

---

5. Code Implementation

Project Structure:

```
fetch-api-project/
│
├── index.html
├── style.css
└── script.js
```

---

index.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Blog & Todo Platform</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <div class="container">
        <h1>Blogging Platform Dashboard</h1>

        <section>
            <h2>Blog Posts</h2>
            <div id="posts">Loading posts...</div>
        </section>

        <section>
            <h2>Todo List</h2>
            <div id="todos">Loading todos...</div>
        </section>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

---

style.css

```css
body {
    font-family: Arial, sans-serif;
    background-color: #f4f6f9;
    padding: 20px;
}

.container {
    max-width: 900px;
    margin: auto;
}

h1 {
    text-align: center;
    margin-bottom: 30px;
}

section {
    margin-bottom: 40px;
}

.card {
    background: #ffffff;
    padding: 12px;
    margin: 10px 0;
    border-radius: 6px;
```

```
    box-shadow: 0 2px 6px rgba(0,0,0,0.1);
}

.completed {
    color: green;
    font-weight: bold;
}

.pending {
    color: red;
    font-weight: bold;
}

.error {
    color: red;
    font-weight: bold;
}
```

---

script.js

```
/*
    Blogging Platform Application
    Demonstrates:
    - Fetch API
    - JSON Handling
    - DOM Manipulation
    - Error Handling
    - Module Pattern
*/

const BlogApp = (function () {

    const POSTS_API = "https://jsonplaceholder.typicode.com/posts";
    const TODOS_API = "https://jsonplaceholder.typicode.com/todos";

    async function fetchData(url) {
        const response = await fetch(url);

        if (!response.ok) {
            throw new Error("Failed to retrieve data from server.");
        }

        return await response.json();
    }

    function displayPosts(posts) {
        const container = document.getElementById("posts");
        container.innerHTML = "";

        posts.slice(0, 5).forEach(post => {
            const div = document.createElement("div");
            div.className = "card";

            div.innerHTML = `
```

```
                <h3>${post.title}</h3>
                <p>${post.body}</p>
            `;

            container.appendChild(div);
        });
    }

    function displayTodos(todos) {
        const container = document.getElementById("todos");
        container.innerHTML = "";

        todos.slice(0, 5).forEach(todo => {
            const div = document.createElement("div");
            div.className = "card";

            div.innerHTML = `
                <p>
                    ${todo.title} -
                    <span class="${todo.completed ? "completed" :
"pending"}">
                        ${todo.completed ? "Completed" : "Pending"}
                    </span>
                </p>
            `;

            container.appendChild(div);
        });
    }

    function showError(containerId, message) {
        document.getElementById(containerId).innerHTML =
            `<p class="error">${message}</p>`;
    }

    async function init() {

        try {
            const posts = await fetchData(POSTS_API);
            displayPosts(posts);
        } catch (error) {
            showError("posts", error.message);
        }

        try {
            const todos = await fetchData(TODOS_API);
            displayTodos(todos);
        } catch (error) {
            showError("todos", error.message);
        }
    }

    return {
        init: init
    };

})();
```

```
BlogApp.init();
```

---

6. Challenges Faced

During the assignment, the following challenges were encountered:

- Understanding asynchronous programming using async/await.
- Handling server responses and validating response status.
- Implementing structured error handling.
- Refactoring the code using the Module Pattern.
- Ensuring proper separation of structure, styling, and logic.

These challenges were resolved through testing and debugging using browser developer tools.

---

7. Testing and Validation

The application was tested under the following conditions:

- Successful API response.
- Network disconnection to simulate server unavailability.
- Invalid API endpoint to test error handling.
- Proper rendering of posts and todos.

All functionalities performed as expected.

---

8. Reflection and Learning Experience

This assignment provided practical exposure to real-world frontend development. It improved understanding of:

- Fetch API and asynchronous JavaScript.
- JSON data handling.
- Dynamic DOM manipulation.
- Implementing robust error handling.
- Structuring scalable applications using design patterns.